



Security Audit

DVX (dApp)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	9
Intended Smart Contract Behaviours	10
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Audit Findings	13
Centralisation	32
Conclusion	33
Our Methodology	34
Disclaimers	36
About Hashlock	37

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The DVX team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

DVX is transforming Web3 trading with a seamless platform for pro traders in perpetual markets. It enables easy, signature-free transactions through hybrid accounts and consolidates cross-margin management, so traders can manage all positions from a single account. DVX also integrates positions across multiple exchanges, providing unified control. By aggregating liquidity from leading Perp DEXs on Arbitrum, it ensures access to the best prices and deepest liquidity, giving traders an edge with optimized, efficient trades. Experience DVX as a powerful, all-in-one trading hub built for the decentralized finance ecosystem.

Project Name: DVX

Compiler Version: 0.8.7

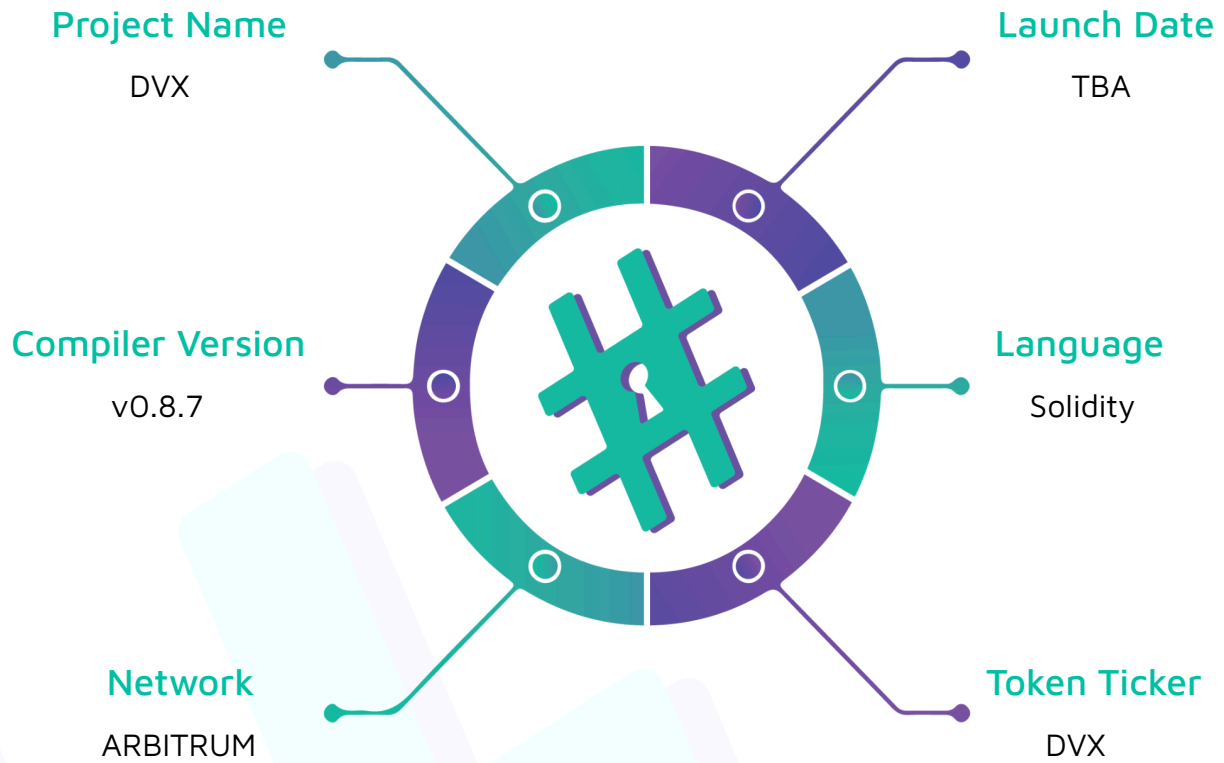
Website: <https://www.dvx.trading/>

Logo:



#Hashlock.

Hashlock Pty Ltd

Visualised Context:

Project Visuals:

THE ULTIMATE PERPETUAL AGGREGATOR FOR PRO TRADERS

Experience one-stop cross margin trading
with multiple perp DEX positions
Built on Arbitrum

FEATURES

SEAMLESS TRADING EXPERIENCE

Proceed the transactions within DVX without additional signatures, using Hybrid Trading Account.

ENJOY CROSS MARGIN

No need to manage margins on each Perp DEXs. Manage your margins in a single DVX account.

Audit scope

We at Hashlock audited the solidity code within the DVX project, the scope of works included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line by line analysis and were supported by software assisted testing.

Description	DVX Protocol Smart Contracts
Platform	Arbitrum / Solidity
Audit Date	June, 2024
Contract 1	Account.sol
Contract 1 MD5 Hash	94f1c73bb8aaadec5a20d99e55bb514c
Contract 2	Exchange.sol
Contract 2 MD5 Hash	999c99eee489371cf9956281625b5591
Contract 3	AccountFactory.sol
Contract 3 MD5 Hash	0960a995070221d27ce2071ac71d9280
Contract 4	AccountProxy.sol
Contract 4 MD5 Hash	ce8d4c1d24513dfc49708e32d8cbdaa1
Contract 5	AccountRouter.sol
Contract 5 MD5 Hash	87aaebd4394f45605d15cfc75e2c8f6b
Contract 6	PayableMulticall.sol
Contract 6 MD5 Hash	5d19eec5048843557c6fdd533dfa5117
Contract 7	AccountV2.sol
Contract 7 MD5 Hash	530691413b7dd01b4c08c327829c1d66
Contract 8	MuxAdapter.sol
Contract 8 MD5 Hash	b1713b4d7f3afc4b105229d19442f7b6

Contract 9	GMXV1Adapter.sol
Contract 9 MD5 Hash	dd47c6ec09047813bb96b5a0b608fd72
Contract 10	Logger.sol
Contract 10 MD5 Hash	5445ef40176d87f8c7ee6012ad9c65d4
Contract 11	MarginManager.sol
Contract 11 MD5 Hash	f2946c067c6915e14ff908dfc7faf36b
Contract 12	Proxy.sol
Contract 12 MD5 Hash	33bef5cc4bcc475db510f1d1620558ad
Contract 13	Quoter.sol
Contract 13 MD5 Hash	cc3edacbe33e12155ed79b60a0db060f
Contract 14	Reader.sol
Contract 14 MD5 Hash	28f6906a76d6bcc3da7189660cbf96cb
Contract 15	Swapper.sol
Contract 15 MD5 Hash	325fd63469c468640a08110a986d09f4
Contract 16	ReferralManager.sol
Contract 16 MD5 Hash	b997ddcd4dad0158246f38934b53a2b7
Contract 17	Warehouse.sol
Contract 17 MD5 Hash	35994cd8d405771b940f0324a8e01d5d
Contract 18	NFTs.sol
Contract 18 MD5 Hash	7f3bb66c7cebc1d92fa9cf31efe513f2
Contract 19	AlphaAccessCard.sol
Contract 19 MD5 Hash	bc8028ba2dbf92477bcf81c7c5887d83
Contract 20	NFT.sol
Contract 20 MD5 Hash	c749852f18f1965cfaea8713633aacac
GitHub Commit Hash	5f4cb3a5df264383ca8704c069469a3610066fcd

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some significant vulnerabilities that have since been addressed.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

5 High severity vulnerabilities

3 Medium severity vulnerabilities

4 Low severity vulnerabilities

4 Gas Optimisations

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Behaviours

Claimed Behaviour	Actual Behaviour
Account.sol - The contract that users interact with - Makes calls to Exchange.sol on behalf of the user - Allows users to: - Create market orders - Interact with their orders - Withdraw or deposit collateral	Contract achieves this functionality.
Exchange.sol - Handles the calls made by Account.sol - Redirects the market order calls to Warehouse.sol - Users will use this contract to create their own accounts - Makes a call to AccountProxy.sol which deploys a new Account.sol for users.	Contract achieves this functionality.
AccountFactory.sol - Handles the createAccount calls made by Exchange.sol - Deploys a new Account.sol contract for the user	Contract achieves this functionality.
Warehouse.sol - Handles calls by Exchange.sol - Main functions are only callable by Exchange.sol - Emits events for users market orders.	Contract achieves this functionality.
Adapters (GMXV1Adapter.sol, MuxAdapter.sol) - Handles the market order calls - Makes calls to DEX's	Contracts achieve this functionality.

Code Quality

This Audit scope involves the smart contracts of the DVX project, as outlined in the Audit Scope section. All contracts, libraries and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is not well commented and does not follow best practice nat-spec styling.

Audit Resources

We were given the DVX project smart contract code in the form of Github access.

As mentioned above, code parts are not well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues and inefficiencies

Audit Findings

High

[H-01] Swapper.sol::Swap - No slippage control on token swap.

Description

There is no slippage control for the users swapping their tokens which will expose users to sandwich attacks.

Vulnerability Details

The `swap()` function has only 3 parameters: `tokenIn`, `tokenOut`, `amountIn`. Users cannot set the expected `amountOutMin` and instead, this value is hardcoded in the function and set to 0.

```
function swap(  
    address tokenIn,  
    address tokenOut,  
    uint256 amountIn  
) public override returns (uint256 amountOut) {  
    IERC20(tokenIn).safeTransferFrom(msg.sender, address(this), amountIn);  
    IERC20(tokenIn).approve(_swapRouter, amountIn);  
    ISwapRouter.ExactInputSingleParams memory params = ISwapRouter  
        .ExactInputSingleParams({  
            tokenIn: tokenIn,  
            tokenOut: tokenOut,  
            fee: 3000,  
            recipient: msg.sender,
```

```

        deadline: block.timestamp,

        amountIn: amountIn,

        amountOutMinimum: 0,

        sqrtPriceLimitX96: 0

    });

    amountOut = ISwapRouter(_swapRouter).exactInputSingle(params);
}

```

Since the value of `amountOutMin` is hard coded to 0 in the function above, users have no slippage protection.

Impact

Users that swap their tokens with this function will be vulnerable to sandwich attacks and they might receive fewer tokens than expected while the pools are volatile.

Recommendation

Add a new parameter to the function so the users can input the minimum amount of tokens they expect to be returned.

Status

Resolved

[H-02] GMXV1Adapter.sol::getPrice - Hardcoded price value for stable tokens

Description

The `getPrice()` function returns `1e30` for any stable coin.

Vulnerability Details

The `getPrice()` function is used to get the price of tokens based on if the token is a stable coin that is added to the `isStableToken` mapping in `Exchange.sol` or if the user's position is a long position.

```
uint256 public constant PRICE_DECIMALS = 30;

uint256 public constant USD = 1 * (10 ** PRICE_DECIMALS);

...

function getPrice(
    address token,
    bool isLong
) public view override returns (uint256) {
    if (IExchange(_exchange).isStableToken(token)) {
        return USD;
    }
    return
        isLong
            ? IVault(_vault).getMaxPrice(token)
            : IVault(_vault).getMinPrice(token);
}
```

As seen in the function above, if the token is in the `isStableToken` mapping, this function will return the hardcoded `USD` value which is hardcoded to be `1e30`. This hardcoded value does not consider that stable coins can de-peg.

Impact

Hardcoding the value of tokens will have an unintended impact during volatile markets. This is especially dangerous if the stable coin de-pegs and rapidly loses value. Many functions in this contract that rely on the price returned by the `getPrice()` function will be returning incorrect prices.

Recommendation

Always check the current price of the tokens even if they are stable coins.

Note

The DVX team knows that the depegging of a stablecoin is significant, but they believe that the price of USDC will remain stable in any circumstance. Therefore, there will be no changes to the codebase.

Status

Acknowledged

[H-03] Account.sol - delegatecall made to unchecked adapter addresses

Description

`Account.sol` contract makes delegatecalls to unchecked adapter addresses.

Vulnerability Details

Functions `decreaseCollateral` and `_decreasePosition` do not have the necessary `isRegisteredAdapter` checks that are done in functions such as `_increaseCollateral`.

```
function _increasePosition(...) internal {  
    require(  
        IExchange(_exchange).isRegisteredAdapter(adapter),  
        "adapter: not registered"  
    );  
}
```

delegatecall needs to be used with caution since the contract being called is using the caller's contract storage. The callee contract can alter the caller's contract state (see [Solidity docs](#)).

Impact

Users inputting the incorrect adapter address can also lead to unexpected results, altering the state of their `Account.sol` contract. This could lead to problems such as the user losing ownership of their `Account.sol` contract.

Recommendation

Implement the `isRegisteredAdapter` check that is done in functions such as `_increaseCollateral`, into `decreaseCollateral` and `_decreasePosition` functions.

Status

Resolved

[H-04] Exchange.sol - access control `onlyAccount` modifier in functions are bypassable

Description

`Exchange.sol` contract handles the calls from an `Account.sol` contract for many functions such as `swap()`, `createLimitOrder()`. However, the `onlyAccount` modifier is bypassable.

Vulnerability Details

Some require statements for functions inside the `Exchange.sol` contract are handled in the `Account.sol` contract, an example shown below:

```
function createLimitOrder( ... ) public payable virtual override noReentrant
onlyOwnerOrRelayer {

    ...

    require(

        IExchange(_exchange).isSupportedCollateralToken(collateral),

        "collateral: not supported"

    );

    require(

        IExchange(_exchange).isSupportedIndexToken(index),

        "index: not supported"
```

```

    );

    require(
        collateralAmount <= getWithdrawableBalance(collateral),
        "collateralAmount: greater than withdrawable balance"
    );

    if (networkFee > 0) {
        require(
            collateralAmount >= networkFee,
            "collateralAmount: less than network fee"
        );

        _collectNetworkFee(collateral, networkFee);

        collateralAmount -= networkFee;
    }

    require(
        executionFee <= collateralAmount,
        "executionFee: greater than collateralAmount"
    );

    ...

```

In the intended use, users have to interact with the `Account.sol` contract and cannot directly interact with the `Exchange.sol` contract due to the implemented modifier. This means that users need to pass the required checks that are implemented in the `Account.sol` contract.

However, the `onlyAccount` modifier is bypassable, allowing a malicious user to interact directly with the `Exchange.sol` contract and bypass the required checks even if they are not able to. This is due to how the modifier is implemented:

```
modifier onlyAccount(address account) {  
  
    require(IAccount(account).beacon() == address(this), "invalid account");  
  
    _;  
  
}
```

As seen above, this modifier makes a call to the msg.sender's `beacon` function and checks if the result is the `Exchange.sol` contract's address.

Malicious users can deploy a smart contract that will fulfill the above check and bypass the modifier, therefore directly interacting with the `Exchange.sol` contract.

Impact

Malicious users bypassing this modifier can also bypass any checks that are in the `Account.sol` contract and interact with the `Exchange.sol` contract directly. Malicious users this way for example can avoid paying protocol fees, use unsupported tokens and bypass the collateral check.

Recommendation

Change the logic of the `onlyAccount` modifier so that only the authorised contract can call the functions in the `Exchange.sol` contract.

Status

Resolved

[H-05] Account.sol - Insecure Signature Verification Scheme

Description

The `Account.sol` contract uses a custom signature verification scheme that does not follow the EIP-712 standard. This can lead to multiple vulnerabilities including replay attacks, cross-chain signature reuse, and possible phishing attacks.

Vulnerability Details

1) Signature Replay Attack

The `_verifySignature` function does not include a nonce in the signed data. Without a nonce, the same signature can be replayed multiple times by relayers to call functions repeatedly.

```
function _verifySignature(  
    uint256 deadline,  
    bytes32 messageHash,  
    bytes memory signature  
) internal view returns (bool) {  
    require(deadline >= block.timestamp, "deadline: expired");  
    require(  
        _delegatedAccountExpiration > block.timestamp,  
        "delegatedAccount: expired"  
    );  
    return  
        _recoverSigner(_getEthSignedMessageHash(messageHash), signature) ==  
        _delegatedAccount;  
}
```

2) Cross-Chain Signature Reuse

Since the signed data does not include the chain ID, signatures valid on one blockchain can be reused on another blockchain, leading to unauthorized actions across different chains.

3) Phishing Attacks

The generic format of the signatures makes it possible for attackers to trick users into signing data that could be reused maliciously on the [Account.sol](#) contract or other applications with similar signature structures.

Impact

The vulnerabilities in the signature verification scheme can lead to unauthorized opening or closing of positions, cross-chain manipulation, and phishing attacks leading to loss of control over the account.

Recommendation

A nonce can be maintained; the value of which can be incremented for every successful signature validation. Ideally, signatures should be created according to the [EIP712](#) standard.

Note

The Hashlock team has seen that the Replay attack is fixed. However, vulnerability to phishing attacks persists as EIP712 has not been implemented.

Status

Acknowledged

Medium

[M-01] AlphaAccessCard.sol - Can airdrop tokens into addresses that are not whitelisted, and does not set the status of the token to claimed.

Description

`airdrop()` function handles the minting of tokens to multiple addresses in one call. However, this function lacks the `whitelist` and `claimed` checks that are done in the `claim` function that handles minting.

Vulnerability Details

Not enforcing these checks, in cases of incorrect owner inputs, can lead to users who are not whitelisted getting tokens that they should not be able to. Not checking if that token is claimed for that user can result in minting another token with the same ID for the user.

Impact

Tokens might get minted for users who should not be getting them. Users can have duplicate tokens minted.

Recommendation

Implement the `whitelist` and `claimed` checks in the for loop inside the `airdrop()` function. An example is shown below:

```
function airdrop(
    address[] calldata accounts,
    uint256 id
) external onlyOwner {
    for (uint256 i = 0; i < accounts.length; i++) {
        require(!_whitelist[accounts[i]][id], "not whitelisted");
        require(!_claimed[accounts[i]][id], "already claimed");
    }
}
```

```

        _mint(accounts[i], id, 1, "");
    }
}

```

Status

Resolved

[M-02] Contracts - Functions should approve(0) first

Description

Some tokens do not work when changing the allowance from an existing non-zero allowance value. They must first be approved by zero and then the actual allowance must be approved.

Vulnerability Details

One of the examples of this is USDT in mainnet, the function shown below:

```

function approve(address _spender, uint _value) public onlyPayloadSize(2 * 32) {

    // To change the approve amount you first have to reduce the addresses`
    // allowance to zero by calling `approve(_spender, 0)` if it is not
    // already 0 to mitigate the race condition described here:
    // https://github.com/ethereum/EIPs/issues/20#issuecomment-263524729

    require(!((_value != 0) && (allowed[msg.sender][_spender] != 0)));

    allowed[msg.sender][_spender] = _value;

    Approval(msg.sender, _spender, _value);

}

```

There are 12 instances of this vulnerability in the following contracts: [Exchange.sol](#), [Swapper.sol](#), [MuxAdapter.sol](#), [Account.sol](#)

#Hashlock.

Impact

This oversight will result in approval calls for such tokens to revert unexpectedly.

Recommendation

When developing functions that update allowances for ERC20 tokens, especially when interacting with tokens known to require this pattern, implement a two-step approval process. For example, this can be done with the function shown below:

```
function resetAndApprove(address token, address spender, uint256 amount) public {  
    IERC20(token).approve(spender, 0);  
    IERC20(token).approve(spender, amount);  
}
```

Note

The DVX team will not utilize ERC20 tokens that require a two-step approval process.

Status

Acknowledged

[M-03] MuxAdapter.sol - Avoid the use of deprecated Chainlink functions

Description

`getPositionFee`, `getFundingFee`, `getPrice`, `_getPrice` functions use a deprecated version of Chainlink functions, consider following the [documentation](#) to use the supported alternatives.

Vulnerability Details

Functions mentioned above use the `latestAnswer` function to get the latest price data from Chainlink oracles. However, this function is deprecated.

Recommendation

Avoid using deprecated Chainlink functions. Refer to the [Chainlink documentation](#) to identify and use the supported alternatives for the functions you require. An example is the `getRoundData` function.

Status

Resolved

Low

[L-01] Account.sol - User can interact with tokens that are not supported by the protocol

Description

Functions such as `_increasePosition` have the `isRegisteredCollateral` check done so that users can only use tokens that are supported by the protocol. However, this check is not implemented in the `_decreasePosition` function.

Recommendation

Implement the `isRegisteredCollateral` check in the `_decreasePosition` function.

Note

The DVX team thinks this is intended behavior, as the `[collateral, index]` pair can be modified by certain adapters. For this reason, they will not restrict the token address when decreasing a position.

Status

Acknowledged

[L-02] MarginManager.sol - Functions always return true

Description

Functions `validateAddAcmmMargin` and `validateSubAcmmMargin` and have no logic implemented in them. These functions are hardcoded to always return true.

Recommendation

Implement the intended functionality into these functions or change the calls made to these functions to just return true in the `Exchange.sol` contract to save gas.

Note

The DVX team clarifies that `MarginManager.sol` has not been implemented or used yet.

Status

Acknowledged

[L-03] Contracts - Initializers can be front-run

Description

Initializers could be front-run, allowing an attacker to either set their own values, take ownership of the contract, and in the best case force a re-deployment.

Recommendation

Arbitrum chain works on a first come, first serve basis but it is best practice to implement access control to initializer functions.

Note

The DVX team will use OpenZeppelin's `ERC1967Proxy.sol` contract, which supports initializing the proxy within its constructor. This allows them to avoid frontrunning, as the proxy contract is both deployed and initialized in a single transaction.

Status

Resolved

[L-04] PayableMulticall.sol - Payable multicall may lead to lost ETH in future

Description

`multicall()` function is declared payable. Multiple delegated-to functions could use the same `msg.value`, which could result in ETH balance calculation errors in the future.

```
function multicall(  
    bytes[] calldata data  
) external payable virtual returns (bytes[] memory results) {  
    results = new bytes[](data.length);  
  
    for (uint256 i; i < data.length; i++) {  
        (bool success, bytes memory result) = address(this).delegatecall(  
            data[i]  
        );  
        require(success, "call failed");  
        results[i] = result;  
    }  
    return results;  
}
```

Recommendation

Consider adding a check for `msg.value` to be future-proof and extra safe. Alternatively, make a comment about the risk.

```
function multicall(  
    bytes[] calldata data  
) external payable virtual returns (bytes[] memory results) {  
    results = new bytes[](data.length);
```

```
require(msg.value == 0);  
for (uint256 i; i < data.length; i++) {  
    (bool success, bytes memory result) = address(this).delegatecall(  
        data[i]  
    );  
    require(success, "call failed");  
    results[i] = result;  
}  
  
return results;  
}
```

Note

The DVX team is carefully using `multicall()` to execute only a single payable function alongside other non-payable functions. The team also considers that the given recommendation is not applicable, as they cannot restrict `msg.value` to zero. Instead, DVX has added a warning comment to the function for clarification.

Status

Resolved

Gas

[G-01] Account.sol, AccountV2.sol - Revert String Size Optimization

Description

Shortening the revert strings to fit within 32 bytes will decrease deployment time and decrease runtime Gas when the revert condition is met.

Revert strings that are longer than 32 bytes require at least one additional `mstore`, along with additional overhead to calculate memory offset, etc.

Recommendation

To optimise Gas usage, it is recommended to keep revert strings as short as possible and to ensure that they fit within 32 bytes. It is possible to use abbreviations or simplified error messages to keep the string length short. Doing so can reduce the amount of Gas used during deployment and runtime when the revert condition is met.

Status

Resolved

[G-02] Account.sol, AccountProxy.sol - Internal functions only called once can be inlined to save gas

Description

If an internal function is only used once, there is no need to modularize it, unless the function calling it would otherwise be too long and complex. Not inlining costs 20 to 40 gas because of two extra JUMP instructions and additional stack operations needed for function calls.

Examples of these functions are: `Account.sol::_collectExecutionFee`, `AccountProxy.sol::_getBeacon`, `AccountProxy.sol::_getVersion`

Recommendation

Inline 'internal' functions in Solidity that are called only once to save gas.

Note

The DVX team clarifies that such functions are modularized for readability and brevity rather than for gas optimization.

Status

Acknowledged

[G-03] AccountFactory.sol::onlyExchange - Inline modifiers used only once**Description**

In Solidity, modifiers provide a way to add reusable conditions or logic to functions. However, if a modifier is used only once, the modularization may not be beneficial in terms of gas efficiency. The overhead associated with a modifier – including two extra JUMP instructions and additional stack operations for the function call – results in an extra cost of about 20 to 40 gas. In cases where a modifier is unique to a single function and the function's complexity does not necessitate breaking out logic, inlining the modifier's code directly within the function can be more gas-efficient. This approach avoids the overhead of a separate modifier call while maintaining code clarity.

Recommendation

Consider inlining the modifier logic within the function itself to save gas. This is especially advantageous for simpler functions where the additional clarity provided by a separate modifier does not outweigh the gas cost of its use.

Status

Resolved

[G-04] ReferralManager.sol::onlyAccount - Unused Modifier**Description**

A modifier is declared in the contract but never used.

Recommendation

Remove redundant modifier `ReferralManager.sol::onlyAccount`

Status

Resolved

Centralisation

The DVX project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlocks analysis, the DVX project seems to have a sound and well tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. However, the code is not well commented. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits are to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and whitebox penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contracts details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment and functionality (performing the intended functions).

Due to the fact that the total number of test cases are unlimited, the audit makes no statements or warranties on security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bugfree status or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds, and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3 oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#Hashlock.

#Hashlock.

Hashlock Pty Ltd